



# Test Account & Seed Data Specification

Multi-Shop Scenarios, Serialized Inventory Fixtures & Persona Sets

| DOCUMENT TYPE    | TARGET AUDIENCE                | RELEASE VERSION   | STATUS                |
|------------------|--------------------------------|-------------------|-----------------------|
| QA Specification | QA Engineers, Automation Teams | v1.4.0 Production | ✔ Verified & Approved |

## Test Account and Data Specification

This document outlines the required test accounts and the essential test data set needed to comprehensively validate the StockWhisk platform, particularly focusing on multi-tenancy, role-based access control, and complex business logic.

### Test Accounts

The following test accounts should be provisioned to verify different roles and permissions across the system. Ensure all accounts have a known, secure test password.

1. Platform Super Admin

- **Email:** admin@stockwhisk.com
- **Role:** Full platform access, Django admin access.
- **Purpose:** System-wide configuration, tenant management.

2. Shop Owner (Test Shop 1)

- **Email:** owner@testshop1.com
- **Role:** Full access to Test Shop 1.
- **Purpose:** Managing shop settings, viewing all reports, managing staff.

3. Shop Manager (Test Shop 1)

- **Email:** manager@testshop1.com
- **Role:** Configurable permissions within Test Shop 1 (e.g., inventory management, reporting, but no shop settings).
- **Purpose:** Testing RBAC and limited administrative capabilities.

4. Cashier (Test Shop 1)

- **Email:** cashier@testshop1.com
- **Role:** POS-only access, daily shift management.
- **Purpose:** Testing everyday sales workflows and preventing unauthorized access to backend modules.

---

## 5. Shop Owner (Test Shop 2)

- **Email:** owner@testshop2.com
- **Role:** Full access to Test Shop 2.
- **Purpose:** Crucial for testing cross-tenant data isolation (ensuring Shop 1 cannot see Shop 2's data).

## 6. Demo Shop User

- **Email:** demo@stockwhisk.com (or similar)
- **Role:** Read-only access to a specific demo tenant.
- **Purpose:** Safe environment for potential clients to explore the platform without altering data.

## 7. Reseller

- **Email:** reseller@test.com
- **Role:** Access to the Reseller portal.
- **Purpose:** Testing tenant creation and management by authorized resellers.

---

## Test Data Requirements

To effectively simulate real-world usage and test all features, the database must contain the following minimum data set:

### Core Entities

- **Tenants (Shops):** At least 2 distinct active shops (to verify data isolation).
- **Categories:** 5+ product categories (e.g., Electronics, Accessories, Services).
- **Brands:** 5+ distinct brands.

### Catalog & Inventory

- **Products:** 50+ diverse products distributed across categories/brands.
  - *Standard Inventory:* Items with track\_inventory=True.
  - *Services/Non-inventory:* Items with track\_inventory=False (e.g., Labor charge, generic accessory).
  - *Serialized Items:* Products utilizing ProductUnit models for individual serial number tracking.
- **Stock Levels:**
  - Items with 0 stock (Out of stock testing).
  - Items with low stock (<5 units, for reorder alert testing).
  - Items with normal/abundant stock.

### CRM (Customers & Suppliers)

- **Customers:** 10+ customers.
  - Mix of customers with zero balance, positive due balances, and advanced payments.
- **Suppliers:** 5+ suppliers.
  - Mix of suppliers with zero payables and outstanding payables.

## Transactions

- **Sales Invoices:** 20+ invoices.
  - Status mix: `PAID` , `PARTIAL` (with partial payments recorded), `DUE` .
  - Include various payment methods (Cash, Card, Bank Transfer).
- **Purchase Orders (POs):** 5+ POs.
  - Status mix: `RECEIVED` (fully added to inventory), `PENDING` (awaiting receipt).
- **EMI/Installments:** Active EMI schedules with recorded and pending installments.

## Service & Maintenance

- **Service Tickets:** Tickets in various lifecycle stages (e.g., `RECEIVED` , `IN_PROGRESS` , `REPAIRED` , `DELIVERED` ).
- **Warranties:**
  - Active warranties.
  - Warranties expiring soon (within 30 days).
  - Expired warranties.

## Accounting & Financials

- **Expenses:** Expense records spanning various expense categories (e.g., Rent, Utilities, Salary).
- **Shifts/Settlements:**
  - Closed daily settlements (historical shift data).
  - An open active shift (for ongoing POS testing).
- **Ledger:** `LedgerEntry` records correctly reflecting all transactions across all account types (Assets, Liabilities, Equity, Revenue, Expenses).

## Seed Data Strategy

To ensure consistency in testing environments, seed data should be managed programmatically.

### Implementation Plan:

1. **Management Commands:** Develop custom Django management commands (e.g., `python manage.py seed_test_data` ) to generate the required data programmatically using factories (like FactoryBoy) or predefined datasets.
2. **Fixtures:** Alternatively, maintain JSON fixtures for core, immutable data (e.g., default permissions, standard categories).
3. **Documentation:** The specific command or script to load this seed data must be documented in the main repository's README for developer onboarding.